OppRadar

TECHNICAL DOCUMENTATION

Technical datasheet

System architecture, technology stack and AI pipeline of the OppRadar platform.

From: OppRadar Team
Date: 29 October 2026
Version: 1.0

Document	Technical datasheet
Product	OppRadar: an AI agent that finds, checks and matches opportunities (scholarships, jobs, fellowships) to each student
Version	1.0
Date	29 October 2026
Author	OppRadar Team
Scope	Web app (frontend and backend), data layer, AI package, and the contract between them

1 Overview

OppRadar discovers opportunities on the web, structures and verifies them once for everyone, then scores each one against every student's profile and explains why it fits. This datasheet describes how the system is built, which technologies it uses and how the work is split between its parts.

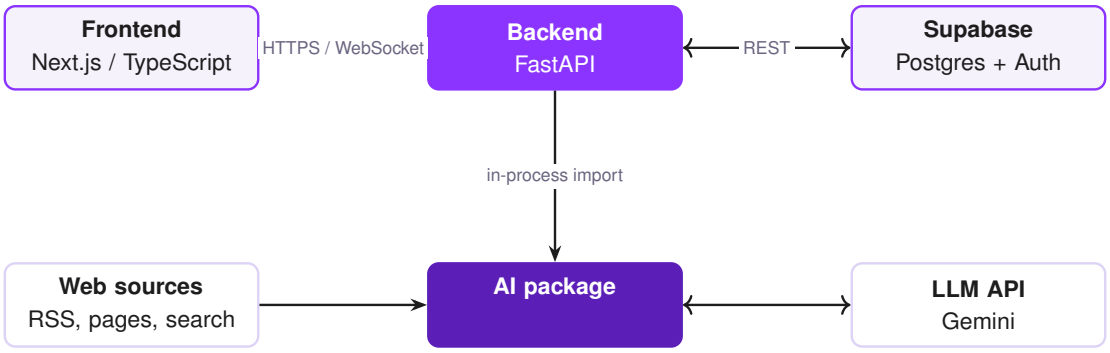
Key design decisions

- **Three tiers, one deployable AI.** Frontend, backend and database are separate. The AI is a plain Python package imported by the backend, so there is no AI microservice to deploy.
- **A narrow contract.** Six functions are the whole interface between the backend and the AI package.
- **Deterministic first, LLM second.** Rules do the filtering, scoring and scam detection. The LLM is used only where rules cannot read messy text.
- **Privacy by default.** User documents never go to a remote model without explicit consent, and the model never sees the profile during matching or chat.
- **One LLM client.** Keys, retries and rate limits live in a single place.

2 Technology stack

Layer	Technology	Role
Frontend	Next.js (App Router), TypeScript	Server-rendered pages; talks only to the backend API
	React Query	Browser-side data fetching through `lib/api/client.ts`, which attaches the Supabase access token
	next-intl	English, French and Arabic; Arabic is right-to-left with logical CSS properties
Backend	FastAPI (Python), Pydantic	REST API and WebSocket for live updates; auth checks and persistence
Data	Supabase: Postgres, Auth	Storage of all rows; Row Level Security (RLS) on every table
AI	`oppradar_ai` Python package	Ingestion, extraction, matching, notifications and chat
	sentence-transformers	Local embeddings for semantic similarity
	Gemini API	Extraction, wording of reasons, chat answers, vision fallback
	Tavily	Web search source for discovery
	rapidocr	Local OCR for scanned documents and screenshots
Config	pydantic-settings, `.env`	All secrets and gates come from the environment, never from code
Hosting	Vercel, Render, Supabase	Frontend, API and database

3 System architecture

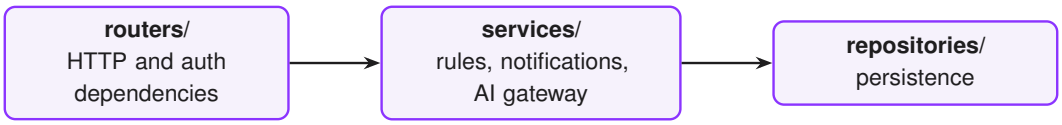


How the parts talk

- The **frontend** talks only to the backend. It never touches the service-role key.
- The **backend** owns authentication checks and persistence. It is the only component that reads and writes Supabase.
- The **AI package** has no database and no web framework. It takes plain Pydantic objects and returns plain Pydantic objects; saving them is the backend's job.
- Everything that needs an LLM goes through the single shared client in `llm.py`.

4 Backend design

The backend (`backend/app`) is layered, and each layer only calls the one below it.



- **One door to the AI.** Routers never import `oppradar_ai`. Only `services/ai.py` does, so each new AI phase is a one-file change.
- **Two interchangeable repositories** implement the same protocol: `SupabaseRepository` (service-role client, one per thread, every user query filters on `user_id`) and `MemoryRepository` (tests and runs without a database).
- **Access levels** come from the Supabase JWT:

Level	Meaning
`User`	Any signed-in user
`Writer`	A signed-in user who is not the read-only demo account
`Admin`	`app_metadata.role` is `admin`, set with `scripts/make_admin.py`

4.1 Opportunity lifecycle

1. The AI pipeline (or `scripts/seed.py` for now) upserts published rows.
2. An organisation can also submit one through `/me/submissions`. It is stored hidden with `review_status = 'pending'`.
3. An admin approves it at `/admin`, and the submitter is notified either way.
4. Admins can re-label the trust level (for example, mark a scam), edit or delete.
5. Students only ever see approved rows, enforced both by the repository filters and by RLS.

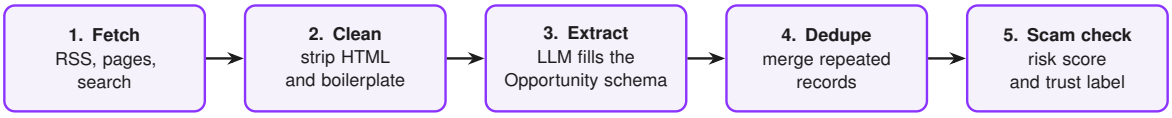
5 Frontend design

- **Rendering.** Next.js App Router renders pages on the server and fetches data in the browser with React Query.
- **Typed API.** Types come from the API's OpenAPI schema (`npm run gen:api`), so nothing is hand-copied.
- **Session.** `middleware.ts` refreshes the session and sends signed-out visitors to `/login?next=...`.
- **Languages.** next-intl with files in `messages/<en|fr|ar>/*.json`; the language is kept in a cookie.
- **Theming.** Colours are theme tokens in `app/globals.css`, redefined under `.dark`.
- **Honest UI.** Screens whose engine waits on an AI phase (match scores, AI chat, drafting, alert delivery) work with what exists and say so. Premium is a free reservation marked "coming soon", with no fake data and no payments.

6 AI pipeline

The AI package has two pipelines that run independently: **ingestion**, which runs once for everyone, and **matching**, which runs per user.

6.1 Ingestion: web sources to clean opportunities

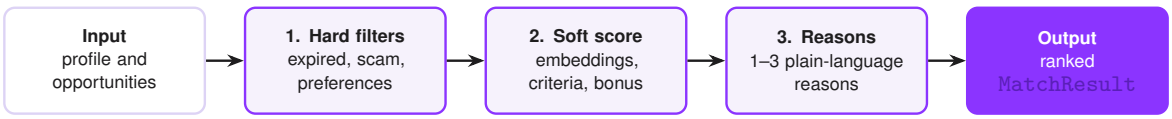


Step	What happens
Fetch	Per-source adapters (RSS, JSON API, HTML list pages, Tavily search) pull raw records. Real User-Agent, timeouts, per-source rate limits, robots.txt honoured. One failing source never stops the run.
Clean	Raw HTML is cleaned before any model sees it.
Extract	An LLM converts each record into the shared `Opportunity` schema in strict JSON mode. Near-structured sources skip the LLM through a fast path. Explicit `Deadline:` lines beat the model, and deadlines are never invented.
Dedupe	URL normalisation first (strip tracking parameters, lowercase host), then fuzzy matching on title and organisation. The freshest deadline and richest description win.
Scam check	Cheap deterministic rules give a 0–1 risk score: requests for money, urgency language, free-mail contacts for a big organisation, no legitimate domain, too-good-to-be-true pay, all-caps or heavy emoji. Above the threshold the item is `scam`; the middle band is `suspicious` and may get an optional LLM second opinion.

Cost control

- One LLM call per **new** record: duplicates are removed before paying.
- RSS entries are **batched**: 5 entries per Gemini call, so the budget of 20 calls per run covers $20 \times 5 = 100$ entries.
- Disk-cache hits are free. A 429 error flips the run to *pending* instead of retrying, and pending items are retried on the next run. Nothing is ever fabricated.
- Free-tier limits: 5 Gemini calls per minute, 5 Tavily searches per run.

6.2 Matching: opportunities to ranked matches



Semantics never override hard constraints:

1. **Hard filters** drop expired and scam-flagged items and anything against the user's declared preferences. Each drop carries a human-readable reason, and *missing data never filters*.
2. **Soft score**, from 0 to 100, for the survivors:

$$\text{score} = 0.5 \times \text{semantic similarity} \; + \; 0.35 \times \text{criteria ratio} \; + \; 0.15 \times \text{bonus}$$

Embeddings are computed locally. Each requirement is marked MET, MISSED or UNKNOWN, with evidence.

3. **Reasons.** Every result carries 1 to 3 `top_reasons` and the `unmet_requirements`, so the app shows an explanation instead of a bare number. The model may only reword the rules' reasons.

6.3 Notifications and chat

Notifications are pure rules, applied in this order:

1. Below the threshold: not a notification at all.
2. Already sent or queued for that opportunity: `suppressed`.
3. Digest mode, quiet hours (22:00 to 08:00 by default) or the daily cap (5 per day): `pending`.
4. Otherwise: `sent`.

Every row keeps a `suppressed_reason`, so the interface can always explain itself. The backend passes stored rows back as `history`, which is how "never twice for the same opportunity" is enforced.

Chat is a thin orchestrator: local evidence (the public record and the requirement verdicts), then a prompt, then one structured model call behind the cost gate. The model never sees the CV. If an answer cannot be grounded, it says exactly "not in the source". A 429 or any model failure degrades to a rules-only answer without retrying.

7 Backend and AI contract

The backend imports the package with `from oppradar_ai import api`. **Six functions are the entire contract.** Everything else inside the package can change freely as long as these signatures and the shared schemas (`ai/oppradar_ai/schemas.py`) hold. Schema changes need approval from both sides.

Function	Input → output	Notes
`parse_documents`	Files → `Profile`	PDF, DOCX, TXT and images; local OCR for scans. Rules-only by default; `allow_llm=True` only with explicit user consent
`run_ingestion`	Sources → `IngestionReport`	One pass of fetch, clean, extract and dedupe. Never raises: failures become report entries
`extract_opportunity`	URL, text or image → `Opportunity`	Exactly one input. A live URL fetch needs the cost gate open
`score_user`	Profile and opportunities → `MatchResult` list	Hard filters, then soft score; sorted best first
`decide_notifications`	Matches, prefs, history → `Notification` list	Pure rules; the backend stores the rows
`answer`	Message, history, opportunity → `ChatAnswer`	Grounded in the public record only

7.1 Error contract

All expected failures are `ApiError` with a machine-readable `code`. The HTTP mapping is the backend's choice (for example `rejected` → 422). Anything else is a bug and may return a 500.

Code	Raised by	Meaning
`empty`	`parse_documents`	No documents given
`unreadable`	`parse_documents`	No file produced any text
`arguments`	`parse_documents`, `extract_opportunity`	Bad document kind, or not exactly one of url, text, image
`offline`	`extract_opportunity`	Live fetch refused (gate closed)
`fetch`	`extract_opportunity`	Page download failed
`rejected`	`extract_opportunity`	The text is not an opportunity
`pending`	`extract_opportunity`	No LLM answer (budget, quota or provider)
`unusable`	`extract_opportunity`	No usable title or organiser
`chat-unavailable`	`answer`	Chat module missing or signature changed

8 End-to-end data flow



1. The scheduler calls `run_ingestion`. The backend saves the validated opportunities.
2. The student uploads a CV once. `parse_documents` turns it into a `Profile`, which the backend saves.
3. `score_user` scores live opportunities against the profile. The backend saves the `MatchResult` rows with their explanations.
4. `decide_notifications` decides what to deliver now, later or never.
5. The frontend reads everything through the backend and shows matches with their reasons. The chat calls `answer` on a chosen opportunity.

9 Security, privacy and cost gates

Data	Rule
User documents (CV, certificates)	Never sent to a remote model by default. Only with `allow_llm=True`, which the backend sets after the user's explicit consent, stored with the profile
Public opportunity pages and feeds	May go to Gemini when the gate is open
Profile during matching and chat	The model never sees it. Embeddings are local
Live web and LLM calls	Need `RUN_LIVE_LLM=1` and a real provider. With `LLM_PROVIDER=fake` (the default) every function works offline on canned data, which is what tests and notebooks use
Secrets	Only in `.env`; the service-role key never reaches the browser
Row access	RLS in Postgres, plus every repository query filtering on `user_id`

10 Evaluation

Contract-first test suites live in `ai/tests/` and run with `pytest ai/tests`. LLM-dependent behaviour is measured on labelled sets in `ai/data/eval/`. Results are appended after every meaningful change.

Eval set	Metric	Labelled data	Latest	Date
Extraction	Field-level F1; exact match on title, organisation, deadline	Raw records with the gold `Opportunity`	TBD	TBD
Scam check	Precision and recall on `scam` (recall weighted higher)	Labelled safe and scam samples	TBD	TBD
Matching	Precision@5 and nDCG@5 per profile	Profile, opportunity, relevance triples	TBD	TBD

11 Current status and limits

- Evaluation numbers are still to be filled in (table above).
- The free LLM tier is rate-limited, so large ingestion runs may leave items *pending* until the next run.
- Premium is a reservation only; there are no payments.
- Screens that depend on an unfinished AI phase say so instead of showing fake data.

OppRadar Team